

Conda Environment

Instalando o MiniConda

Para começar a utilizar ambientes virtuais com Conda, o primeiro passo é instalar o Miniconda no Ubuntu. Antes disso, é recomendável atualizar o sistema para evitar conflitos e garantir que todas as ferramentas estejam em sua versão mais recente. Para isso, utilizamos o comando:

```
sudo apt update && sudo apt upgrade -y
```

Com o sistema atualizado, precisamos baixar o instalador do Miniconda diretamente do site oficial. A forma mais prática é usar o **wget** no terminal, copiando o link da versão mais recente. Um exemplo comum é:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Depois que o arquivo for baixado, é necessário torná-lo executável para que o instalador possa ser iniciado. Isso é feito com o comando:

```
chmod +x Miniconda3-latest-Linux-x86_64.sh
```

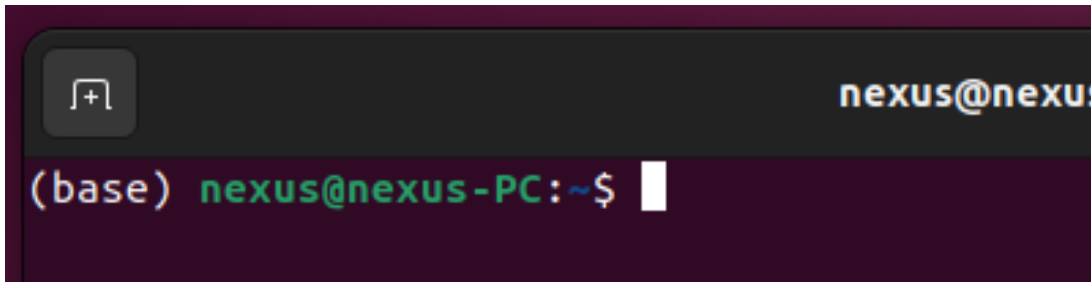
Em seguida, rodamos o instalador. Ao executar esse comando, o Miniconda abrirá um guia de instalação no próprio terminal, pedindo para aceitar a licença e confirmar o local onde será instalado:

```
./Miniconda3-latest-Linux-x86_64.sh
```

Quando a instalação for concluída, é importante reiniciar o terminal ou inicializar o Conda manualmente para ativar o sistema de gerenciamento. Isso pode ser feito com o comando:

```
source ~/.bashrc
```

No seu terminal deve aparecer o **(base)**, isso indica qual é o ambiente python que seu terminal esta utilizando. O base é o ambiente conda principal seu.



Se no seu terminal não aparecer o `(base)`, rode o seguinte comando:

```
conda config --set auto_activate_base true
```

A partir desse momento, o Conda estará disponível no sistema e pronto para criar e gerenciar ambientes virtuais. Para verificar se tudo foi instalado corretamente, basta executar:

```
conda --version
```

Criando o Ambiente Python

Com o Conda corretamente instalado no sistema, o próximo passo é criar seu primeiro ambiente virtual. Um ambiente Conda é como uma pasta isolada onde ficam armazenadas as bibliotecas e a versão específica do Python utilizada naquele projeto. Isso garante organização e evita que diferentes projetos interfiram uns nos outros. Para criar um novo ambiente, utilizamos o comando abaixo, escolhendo o nome e a versão do Python desejada. Neste exemplo, criamos um ambiente chamado “meu_ambiente” com Python 3.10:

```
conda create -n meu_ambiente python=3.10
```

Lembre da aula de linux onde comentamos sobre os parâmetros de comandos. O `-n` é uma abreviação para *name*.

Ativando o ambiente e instalando pacotes

Depois de criado, o ambiente ainda não está ativo. Para entrar nele e começar a trabalhar, precisamos ativá-lo. A ativação faz com que o terminal passe a usar aquele ambiente específico, com suas bibliotecas e configurações. O comando é:

```
conda activate meu_ambiente
```

Lembrete: O ambiente conda só é ativado no terminal que você rodou o comando. A cada novo terminal que deseja utilizar o ambiente tem que rodar o activate novamente.

Com o ambiente ativo, podemos instalar pacotes normalmente usando o Conda ou até mesmo o pip. Por exemplo, se quisermos instalar o NumPy utilizando o próprio Conda, podemos fazer:

```
conda install numpy
```

E, caso um pacote esteja disponível apenas via pip, também é possível instalá-lo sem problemas:

```
pip install nome_do_pacote
```

Quando terminamos de trabalhar em um ambiente, podemos simplesmente desativá-lo. Isso faz o terminal voltar ao ambiente base do Conda, evitando confusão entre projetos diferentes. Para desativar, usamos:

```
conda deactivate
```

Removendo um ambiente

Se um ambiente não for mais necessário, podemos removê-lo completamente para liberar espaço e manter o sistema organizado. Basta garantir que ele esteja desativado e usar o comando

```
conda remove -n meu_ambiente --all
```

Listando ambientes

Por fim, é útil saber como listar todos os ambientes existentes no sistema. Isso ajuda a manter o controle sobre quais ambientes estão sendo usados e quais podem ser removidos. O comando para isso é

```
conda env list
```

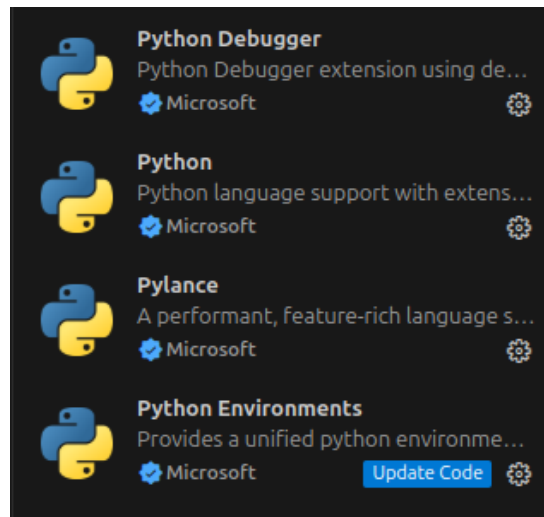
Ou

```
conda info --env
```

Setup do VScode

Vamos utilizar principalmente o VS Code. A principal vantagem desse editor é a enorme variedade de extensões disponíveis, que permitem personalizar o ambiente de acordo com a necessidade e tornam o processo de desenvolvimento muito mais prático.

Ao abrir um arquivo Python pela primeira vez, o próprio VS Code costuma sugerir algumas extensões essenciais. Ainda assim, vale destacar que existe um conjunto de extensões recomendadas que tornam o trabalho com Python mais fluido, desde a execução do código até a navegação entre arquivos, completando o ambiente ideal para quem está iniciando ou já trabalha de forma mais avançada.



Rodando um primeiro Script

No VScode crie um arquivo `meu_primeiro_programa.py`. E vamos rodar nosso o primeiro código universal:

```
print("Hello World")
```